

ProjetDataEngineer

Plateforme Data Engineering End-to-End pour l'analyse e-commerce

Portfolio project — Data Engineering, Analytics Engineering, SQL, BI et Cloud

Python PostgreSQL dbt Apache Airflow Docker
Power BI Streamlit GitHub Actions Render Cloud

Sources : CSV • FakeStore API • Faker • JSON

Ingestion Python : pandas, requests, SQLAlchemy

PostgreSQL : RAW, STAGING, MART, AUDIT

dbt : nettoyage, typage, tests, modèles analytiques

Airflow : orchestration quotidienne, dépendances, retries

Power BI + Streamlit : restitution BI et application analytique

Cloud et CI/CD : Render, Streamlit Cloud, Docker, GitHub Actions

Abdoulaye Diop

Data Engineer Analytics Engineer SQL Engineer

GitHub : <https://github.com/ABDOULAYEDIOP150/ProjetDataEngineer>

Demo : <https://endtoenddiopabdoulaye.streamlit.app>

Mai 2026

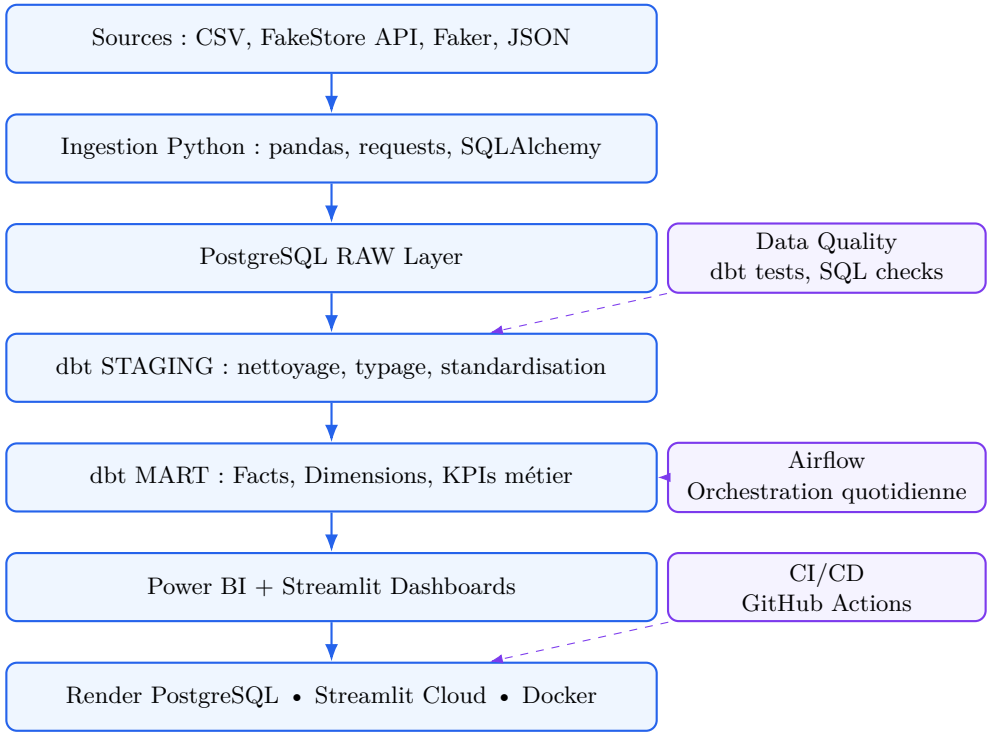
1. Résumé du projet

J’ai construit une plateforme Data Engineering end-to-end autour d’un cas e-commerce. Le but était de relier toutes les étapes d’une chaîne data moderne : récupérer des données depuis plusieurs sources, les charger dans PostgreSQL, les transformer avec dbt, automatiser le pipeline avec Airflow, contrôler la qualité, puis exposer les résultats dans Power BI et Streamlit.

Élément	Réalisation concrète
Cas métier	Simulation d’une activité e-commerce : clients, produits, commandes, lignes de commande et paiements.
Sources	CSV transactionnels, FakeStore API, données Faker et fichiers JSON intermédiaires.
Pipeline	Chaîne ELT automatisée : extraction, chargement RAW, transformations dbt, tests, restitution BI.
Data Warehouse	PostgreSQL structuré en quatre couches : RAW, STAGING, MART et AUDIT.
Modélisation	Schéma en étoile avec tables de faits, dimensions, KPIs métier et mesures DAX.
Industrialisation	Docker, GitHub Actions, Render Cloud, Streamlit Cloud et gestion des secrets.

Un simple dashboard ? Non. Ici, on part des données brutes et on finit par un vrai support pour la décision métier

2. Architecture générale



J’ai choisi une approche ELT : les données sont d’abord chargées brutes dans PostgreSQL, puis transformées dans dbt. Cela permet de garder une trace des données d’origine et de rejouer les transformations sans toucher aux sources.

Technologies utilisées



3. Données et ingestion

Source	Contenu	Type
customers.csv	Clients : identifiant, nom, email, téléphone, ville, pays.	CSV
products.csv	Produits : identifiant, nom, catégorie, prix.	CSV
orders.csv	Commandes : client, date, statut et montant.	CSV
order_items.csv	Lignes de commande : produit, quantité et prix unitaire.	CSV
payments.csv	Paielements : méthode, statut et montant.	CSV
FakeStore API	Produits, utilisateurs et paniers récupérés via API REST.	API
Faker	Données synthétiques d'enrichissement pour élargir les cas de test.	Généré
JSON	Format intermédiaire utilisé pour certaines données API.	JSON

Listing 1 – Extrait simplifié de l'ingestion CSV vers PostgreSQL

```

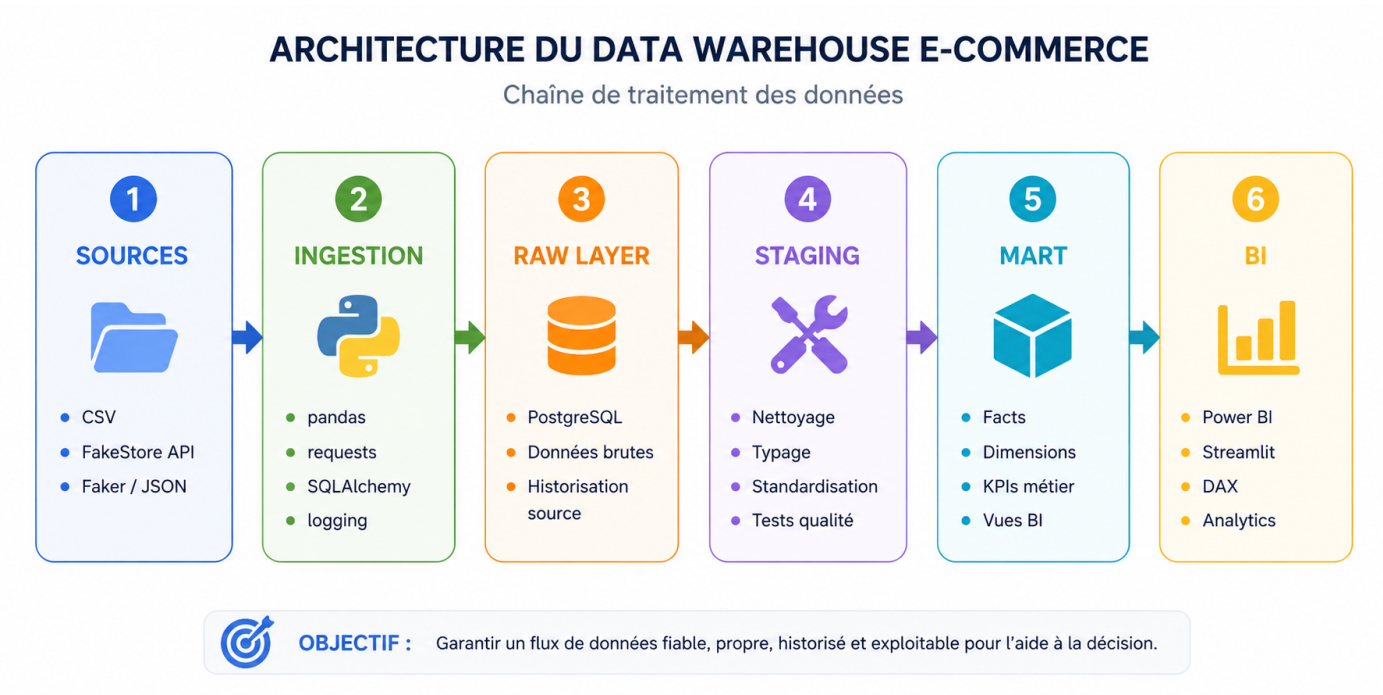
1 from sqlalchemy import create_engine
2 import pandas as pd
3
4 engine = create_engine(
5     f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
6 )
7
8 df = pd.read_csv("data/customers.csv")
9
10 df.to_sql(
11     name="customers",
12     con=engine,
13     schema="raw",
14     if_exists="replace",
15     index=False
16 )

```

La couche RAW garde les données dans leur forme d'origine. Les règles de nettoyage, de typage et de contrôle sont ensuite gérées dans dbt, ce qui rend le pipeline plus lisible et plus simple à maintenir.

4. Data Warehouse PostgreSQL et dbt

Organisation en couches



Fonctionnalités PostgreSQL utilisées

- Window functions : RANK, LAG, LEAD, SUM OVER
 - Vues matérialisées pour Power BI et Streamlit
 - Index composites et partiels
 - Procédures stockées PL/pgSQL
- Triggers SQL
 - Rôles et permissions par schéma
 - Connexions SSL vers Render Cloud
 - EXPLAIN ANALYZE pour l'optimisation

Modèles dbt créés

Couche	Modèles	Statut
Sources	customers, orders, order_items, products, payments	OK
Staging	stg_customers, stg_orders, stg_products, stg_payments, stg_order_items	OK
Mart	fact_sales, fact_payments, dim_customers, dim_products, dim_date	OK

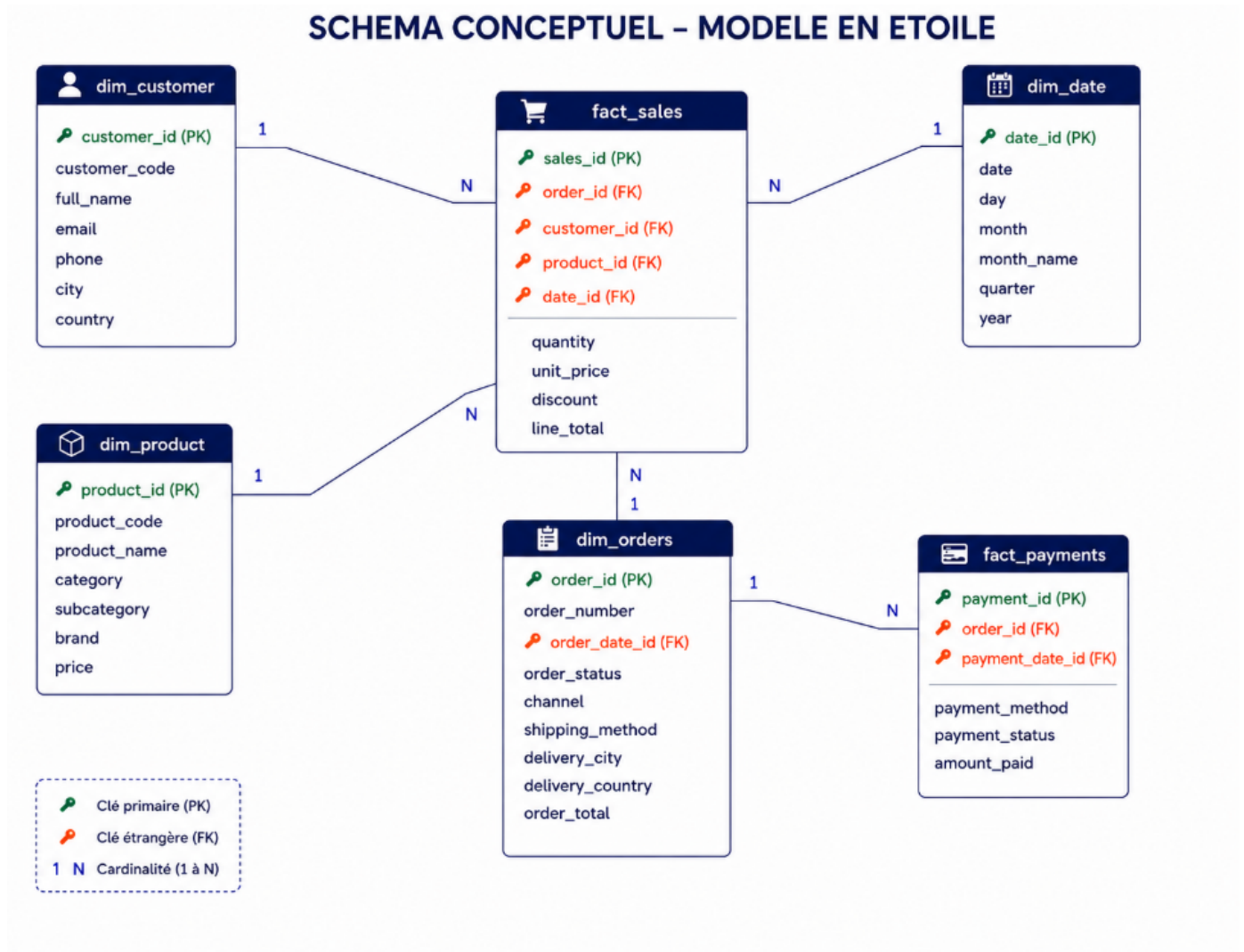
Listing 2 – Extrait du modèle mart fact_sales

```
1 SELECT
2     oi.sales_id,
3     o.order_id,
4     o.customer_id,
5     oi.product_id,
6     o.order_date,
7     o.status,
8     oi.quantity,
9     oi.unit_price,
10    oi.quantity * oi.unit_price AS line_total
11 FROM {{ ref('stg_orders') }} o
12 LEFT JOIN {{ ref('stg_order_items') }} oi
13     ON o.order_id = oi.order_id;
```

5. Modélisation analytique et qualité des données

Modèle analytique

Le modèle BI repose sur un schéma en étoile. J'ai séparé `fact_sales` et `fact_payments` car les deux tables n'ont pas le même grain : une commande peut contenir plusieurs lignes de vente, alors qu'un paiement représente un événement financier distinct.



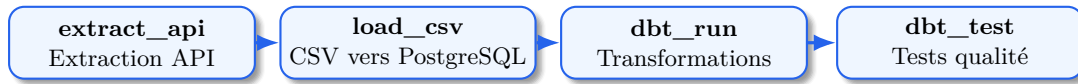
Contrôles qualité

Contrôle	Objectif	Résultat
Doublons commandes	Vérifier l'unicité des commandes.	OK
Clés nulles	Contrôler les identifiants critiques.	OK
Paiements négatifs	Empêcher les montants invalides.	OK
Relations orphelines	Vérifier les liens entre faits et dimensions.	OK
Dates invalides	Contrôler la cohérence chronologique.	OK
Valeurs de statut	Restreindre les statuts aux valeurs métier attendues.	OK

Pendant le développement, certains contrôles ont révélé des valeurs nulles et des relations incomplètes. J'ai corrigé ces points dans la couche STAGING avant d'alimenter les modèles MART.

6. Orchestration Apache Airflow

J'ai utilisé Airflow pour automatiser la séquence complète : extraction API, chargement CSV, exécution dbt, puis tests dbt. Le DAG est planifié en `@daily`, avec deux retries et un délai de cinq minutes pour gérer les erreurs temporaires.



Listing 3 – Extrait du DAG Airflow

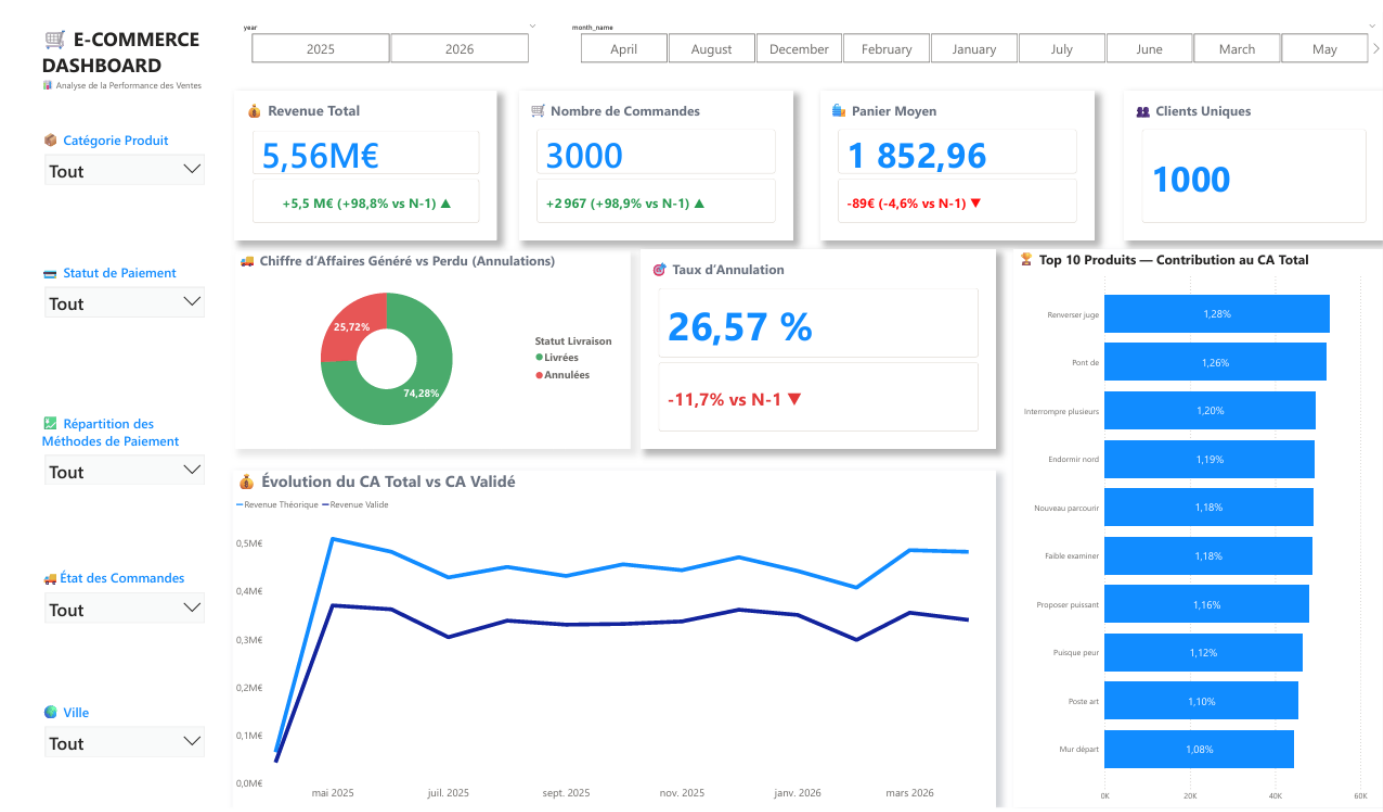
```

1 with DAG(
2     dag_id="ecommerce_data_pipeline",
3     description="Pipeline e-commerce : ingestion + dbt + tests",
4     start_date=datetime(2025, 1, 1),
5     schedule_interval="@daily",
6     catchup=False,
7     default_args=default_args,
8     tags=["ecommerce", "python", "postgresql", "dbt"],
9 ) as dag:
10
11     extract_api = BashOperator(
12         task_id="extract_api",
13         bash_command=f"cd {PROJECT_DIR} && python ingestion/extract_api.py",
14     )
15
16     load_csv_to_postgres = BashOperator(
17         task_id="load_csv_to_postgres",
18         bash_command=f"cd {PROJECT_DIR} && python ingestion/load_csv_to_postgres.py",
19     )
20
21     dbt_run = BashOperator(
22         task_id="dbt_run",
23         bash_command=f"cd {DBT_DIR} && dbt run --profiles-dir /opt/airflow/.dbt",
24     )
25
26     dbt_test = BashOperator(
27         task_id="dbt_test",
28         bash_command=f"cd {DBT_DIR} && dbt test --profiles-dir /opt/airflow/.dbt",
29     )
30
31     extract_api >> load_csv_to_postgres >> dbt_run >> dbt_test
  
```

7. Restitution BI : Power BI et Streamlit

Power BI

Power BI est connecté au Data Warehouse PostgreSQL hébergé sur Render Cloud via le connecteur PostgreSQL natif avec `sslmode=require`. Les vues du schéma `mart` servent de base aux mesures DAX et aux visuels du dashboard.



KPI	Lecture métier
CA total : 5,56 M€	Le chiffre d'affaires progresse fortement, principalement grâce au volume de commandes.
Commandes : 3 000	Le nombre de commandes augmente de 98,9% par rapport à N-1.
Panier moyen : 1 852,96 €	Le panier moyen baisse légèrement, mais l'effet volume compense cette baisse.
Clients uniques : 1 000	La base client permet d'analyser segmentation, récurrence et valeur client.
Taux d'annulation : 26,57%	Indicateur important à suivre pour mesurer la perte potentielle de CA.

Listing 4 – Exemples de mesures DAX

```
1 Revenue Valide =
2 CALCULATE(
3     SUM('mart fact_sales'[line_total]),
4     'mart fact_sales'[status] <> "cancelled"
5 )
6
7 Panier Moyen =
8 DIVIDE([Revenue Valide], DISTINCTCOUNT('mart fact_sales'[order_id]))
9
10 Taux Annulation =
11 DIVIDE(
12     CALCULATE(COUNTROWS('mart v_orders_summary'),
13         'mart v_orders_summary'[status] = "cancelled"),
14     COUNTROWS('mart v_orders_summary')
15 )
```

Streamlit

Demo publique : <https://endtoenddiopabdoulaye.streamlit.app>

L'application Streamlit est organisée en trois espaces : exploration RAW, contrôle STAGING et analyse MART. Elle permet de vérifier rapidement les volumes, les valeurs nulles, les doublons et les KPIs sans repasser par SQL à chaque consultation.

8. Cloud, CI/CD et sécurité

Composant	Mise en place	Statut
PostgreSQL Render	Data Warehouse cloud avec connexion SSL.	OK
Streamlit Cloud	Application analytique publique.	OK
Docker	Image publiée et utilisée pour rendre le projet reproductible.	OK
GitHub Actions CI	Validation Python et exécution des tests dbt.	OK
GitHub Actions CD	Build et push de l'image Docker.	OK
Secrets	Variables sensibles séparées du code : .env, GitHub Secrets, Streamlit Secrets.	OK

Le projet simule une architecture proche d'un environnement de production. Les choix faits autour des couches, des secrets, des tests, de l'orchestration et du déploiement visent surtout à montrer une démarche fiable et reproductible.

9. Problèmes rencontrés et solutions

Sujet	Problème rencontré	Solution mise en place
Connexion cloud	Render impose une connexion sécurisée et une bonne gestion des credentials.	Configuration SSL avec SQLAlchemy et externalisation des variables sensibles.
Organisation DWH	Les données brutes et analytiques pouvaient vite se mélanger.	Séparation RAW, STAGING, MART et AUDIT.
Qualité des données	Valeurs nulles, doublons et relations incomplètes détectés pendant les tests.	Tests dbt, contrôles SQL et corrections dans les modèles STAGING.
Orchestration	Besoin d'automatiser toute la chaîne sans intervention manuelle.	DAG Airflow avec dépendances explicites, retries et logs.
Modélisation BI	Risque d'erreurs de calcul si le grain des tables n'était pas clair.	Séparation des faits, dimensions et mesures DAX dans un modèle en étoile.
Déploiement	Passage d'un projet local à une version consultable en ligne.	Docker, Render, Streamlit Cloud et GitHub Actions.

10. Compétences démontrées et prochaines étapes

Compétences démontrées

- Ingestion multi-sources avec Python
- Modélisation Data Warehouse PostgreSQL
- Architecture RAW / STAGING / MART / AUDIT
- Transformations et tests avec dbt
- Orchestration avec Apache Airflow
- Requêtes SQL analytiques et optimisation
- Modèle en étoile et mesures DAX
- Dashboard Power BI et application Streamlit
- Docker, GitHub Actions et déploiement cloud
- Gestion des secrets et traçabilité des exécutions

Axes d'amélioration

- Ajouter Great Expectations pour renforcer les contrôles qualité.
- Intégrer Kafka pour simuler une ingestion temps réel.
- Déployer l'infrastructure avec Terraform.
- Ajouter un monitoring plus avancé avec alerting.
- Adapter l'architecture vers BigQuery ou Snowflake.

Ce projet m'a surtout permis d'assembler plusieurs briques que j'avais déjà travaillées séparément : ingestion, SQL, modélisation, transformation, orchestration, qualité, BI et cloud. Le résultat est une chaîne data complète, lisible et déployée, qui montre ma capacité à construire un pipeline de bout en bout et à le rendre exploitable côté métier.

Abdoulaye Diop

Data Engineer Analytics Engineer SQL Engineer

GitHub : <https://github.com/ABDOULAYEDIOP150/ProjetDataEngineer>

Demo : <https://endtoenddiopabdoulaye.streamlit.app>

Merci pour votre attention